

❓ Question 1: Why does defining a custom constructor suppress the default constructor in C#?

When we define a custom constructor, C# does not automatically provide the default constructor. This gives the programmer full control over how objects are initialized. If we need a default constructor, we must define it explicitly.

❓ Question 2: How does method overloading improve code readability and reusability?

Method overloading improves code readability by allowing methods with the same name to perform similar operations with different parameters. It also improves reusability because the same method name can be used for different data types or numbers of parameters.

❓ Question 3: What is the purpose of constructor chaining in inheritance?

The purpose of constructor chaining in inheritance is to ensure that the base class constructor is called when creating an object of the derived class. It allows the derived class to initialize the inherited properties and avoids code duplication.

❓ Question 4: How does new differ from override in method overriding?

The new keyword hides the method in the base class, while override replaces the virtual method in the derived class. override supports polymorphism, but new does not.

❓ Question 5: Why is ToString() often overridden in custom classes?

ToString() is often overridden to provide a meaningful string representation of an object. It makes the object's data easier to read and understand when printing or displaying the object.

Part02

1. What is the difference between class and struct in C#?

A **class** is a reference type, while a **struct** is a value type.

- Class objects are stored as references and can support inheritance.
- Structs are value types and do not support class inheritance.
- A class can inherit from another class, while a struct cannot inherit from a class or another struct.
- Structs are usually suitable for small data structures, while classes are suitable for more complex objects.

2. If inheritance is a relation between classes, clarify other relations between classes.

Other important relationships between classes include:

- **Association:** A class uses or is connected to another class.
- **Aggregation:** A class contains another class, but the contained object can exist independently.
- **Composition:** A class contains another class, and the contained object depends on the lifetime of the containing object.
- **Dependency:** A class temporarily depends on another class to perform a specific operation.

Part03 Bonus

1- self study report

1. Parent–Child, Folder–Subfolder, and Modular Monolithic Architecture

In object-oriented programming, the **Parent–Child relationship** is created through inheritance. The Child class inherits common properties and behaviors from the Parent class and can add or extend functionality specific to it.

A similar concept can be seen in project organization through **folders and subfolders** or **namespaces and sub-namespaces**. A subfolder or sub-namespace is placed inside a larger structure to organize more specific and related components.

This idea can also be applied in **Modular Monolithic Architecture**. In this architecture, the whole application is developed and deployed as one application, but its internal

code is separated into independent modules based on different business functions, such as Orders, Inventory, or Billing. Each module can have its own folders and namespaces, such as `Orders.Domain`, `Orders.Services`, and `Orders.Data`.

Therefore, both Parent–Child inheritance and folder–subfolder organization represent a hierarchical structure. This helps keep the code organized, separates responsibilities, and makes different parts of the application easier to understand, maintain, and test. Unlike microservices, however, all modules in a modular monolith remain part of the same application and do not require separate deployments or network communication between them.

2. **new vs override, and Early (Static) vs Late (Dynamic) Binding**

The **new** and **override** keywords allow a Child class to define a method with the same name as a method in its Parent class, but they work differently.

The **new** keyword hides the Parent's method. The method that is executed depends on the **declared type of the reference**. This is known as **Early or Static Binding**, because the method call is determined at compile time.

The **override** keyword is used when the Parent method is declared as `virtual` or `abstract`. It allows the Child class to provide a different implementation of the Parent method. In this case, the method that runs depends on the **actual type of the object at runtime**. This is called **Late or Dynamic Binding**.

For example:

```
Parent obj = new Child();
```

Here, `Parent` is the **reference type**, while `Child` is the **actual object type**.

- With **new**, the Parent method is selected based on the reference type.
- With **override**, the Child method is selected based on the actual object type.

Therefore:

new** → Static/Early Binding → depends on the reference type.**

override** → Dynamic/Late Binding → depends on the actual object type.**

2- what is static and dynamic binding

Binding is the process of determining which method will be executed when a method is called.

Static Binding, also called **Early Binding**, occurs at **compile time**. The compiler decides which method to call based on the declared type of the reference. It is commonly used with normal methods and methods hidden using the `new` keyword.

Dynamic Binding, also called **Late Binding**, occurs at **runtime**. The program determines which method to execute based on the actual type of the object. It is mainly used with `virtual` and `override` methods and is an important part of runtime polymorphism.

For example:

```
Parent obj = new Child();
```

Here, `Parent` is the reference type, while `Child` is the actual object.

- If `new` is used, the method is selected according to the **reference type**, so this is **Static Binding**.
- If `override` is used, the method is selected according to the **actual object type**, so this is **Dynamic Binding**.

In short:

Static Binding → Compile Time → Reference Type

Dynamic Binding → Runtime → Actual Object Type